SISTEMA DE INGENIERÍA AUTÓNOMA

CODEBOT.

Del backlog a una PR verificable.
Arquitectura, funcionamiento y recuperación persistente.

Python + Docker | Claude / OpenCode | OpenSpec + Superpowers | Slack / Email

Basado en CODEBOT.html y el código al 09.10.2026, incluida la recuperación de evidencia de 933a742. No certifica la versión desplegada en Azure.

El agente trabaja. El controlador decide.

Codebot

Supervisa fases, pruebas y resultados. Posee el estado de ejecución, las transiciones y la publicación.

Agente

Explora, propone, implementa, diagnostica y revisa. Produce código, commits locales y contratos.

Persona

Aprueba el acuerdo de trabajo, resuelve decisiones materiales y autoriza el merge de la PR.

No es una sesión de chat que «hace todo»: es un **supervisor persistente de ingeniería**.

02 / MAPA DE LA PRESENTACIÓN

Tres perspectivas de un mismo sistema

Estructura

Componentes, entidades, adaptadores, runtime y supervisores auxiliares.

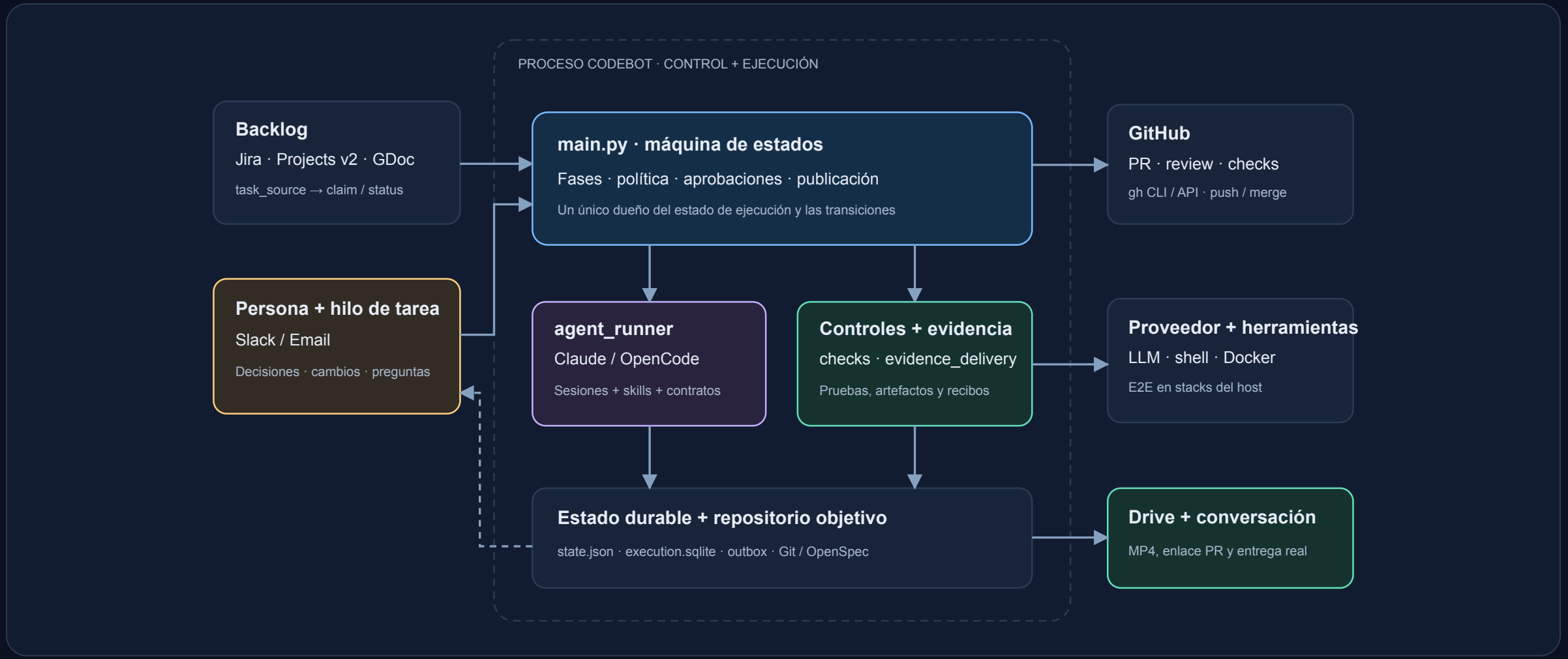
Ejecución

OpenSpec, Superpowers, ciclo de tarea, FSM y flujo dinámico.

Confianza

Decisiones, checks reales, evidencia, recibos y recuperación.

Integraciones intercambiables, un controlador central



La fuente de tareas, el canal de conversación y el agente se eligen de forma independiente.

Qué entra y qué sale de Codebot

Entrada de trabajo

Ticket o issue, detalles e imágenes; contexto técnico y decisiones del mismo hilo.

Claims y holds usan identidad de instancia y fingerprint persistentes.

Salida verificable

Propuesta, implementación, resultados reales, PR, vídeo o reporte y confirmaciones de entrega.

El backlog conserva actividad y el enlace a la PR.

Codebot vive en su propio repositorio y trabaja sobre el checkout indicado por CODEBOT_REPO_PATH.

04 / OPENSPEC + SUPERPOWERS

Acuerdo, método y autoridad

OpenSpec

Qué construir y por qué.

Requisitos, diseño y tareas inspeccionables; contexto que sobrevive a una sesión.

Superpowers

Cómo construirlo.

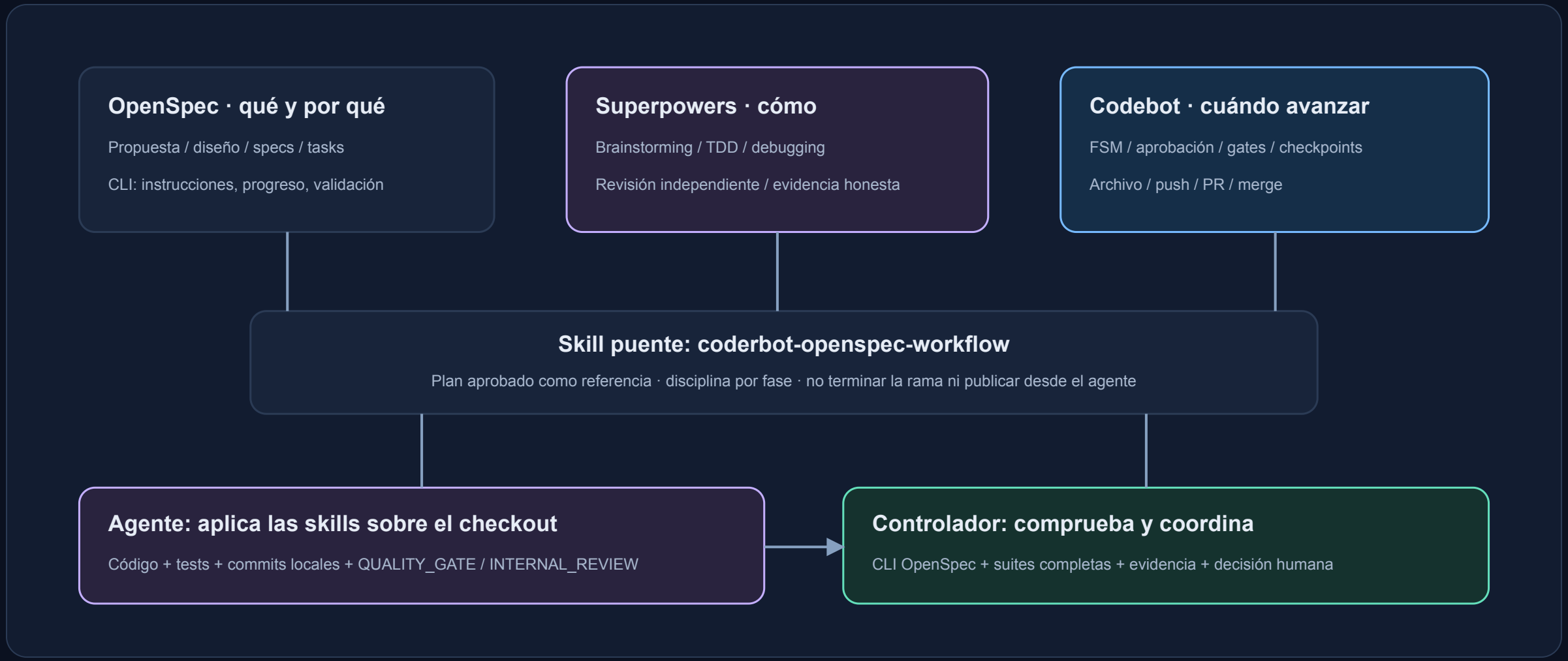
Brainstorming, TDD, diagnóstico sistemático y revisión independiente.

Codebot

Cuándo avanzar.

Fases, aprobación, gates, checks, archivo, push, PR y merge.

OpenSpec y Superpowers no son controladores rivales



La skill coderbot-openspec-workflow mantiene la frontera entre disciplina de ingeniería y autoridad de orquestación.

OpenSpec conserva el acuerdo aprobado

```
openspec/
├── changes/<slug>/
│   ├── proposal.md       Motivación y alcance
│   ├── design.md         Diseño técnico y riesgos
│   ├── specs/…/spec.md   Requisitos y escenarios
│   └── tasks.md          Checklist de trabajo
├── specs/…/spec.md       Especificaciones consolidadas
└── changes/archive/…/   Historia del cambio
```

Cambiar checkmarks no es cambiar alcance. Añadir o quitar tareas puede requerir una **nueva decisión de producto**.

Superpowers influye en cada fase de ingeniería

Explorar e implementar

`brainstorming`: opciones, riesgos y diseño simple.

`test-driven-development`: prueba RED, implementación GREEN y evidencia honesta.

Diagnosticar y revisar

`systematic-debugging`: causa antes de editar.

`requesting-code-review`: revisor fresco, corrección de hallazgos y re-review.

Invocar una skill no demuestra cumplimiento. Los comandos, resultados y controles reales siguen siendo necesarios.

De las skills a las puertas del controlador

Integración gestionada

La imagen fija OpenSpec 1.9.0 y Superpowers v6.3.0.

Genera skills para Claude y OpenCode; los runners cargan plugins y rutas gestionadas.

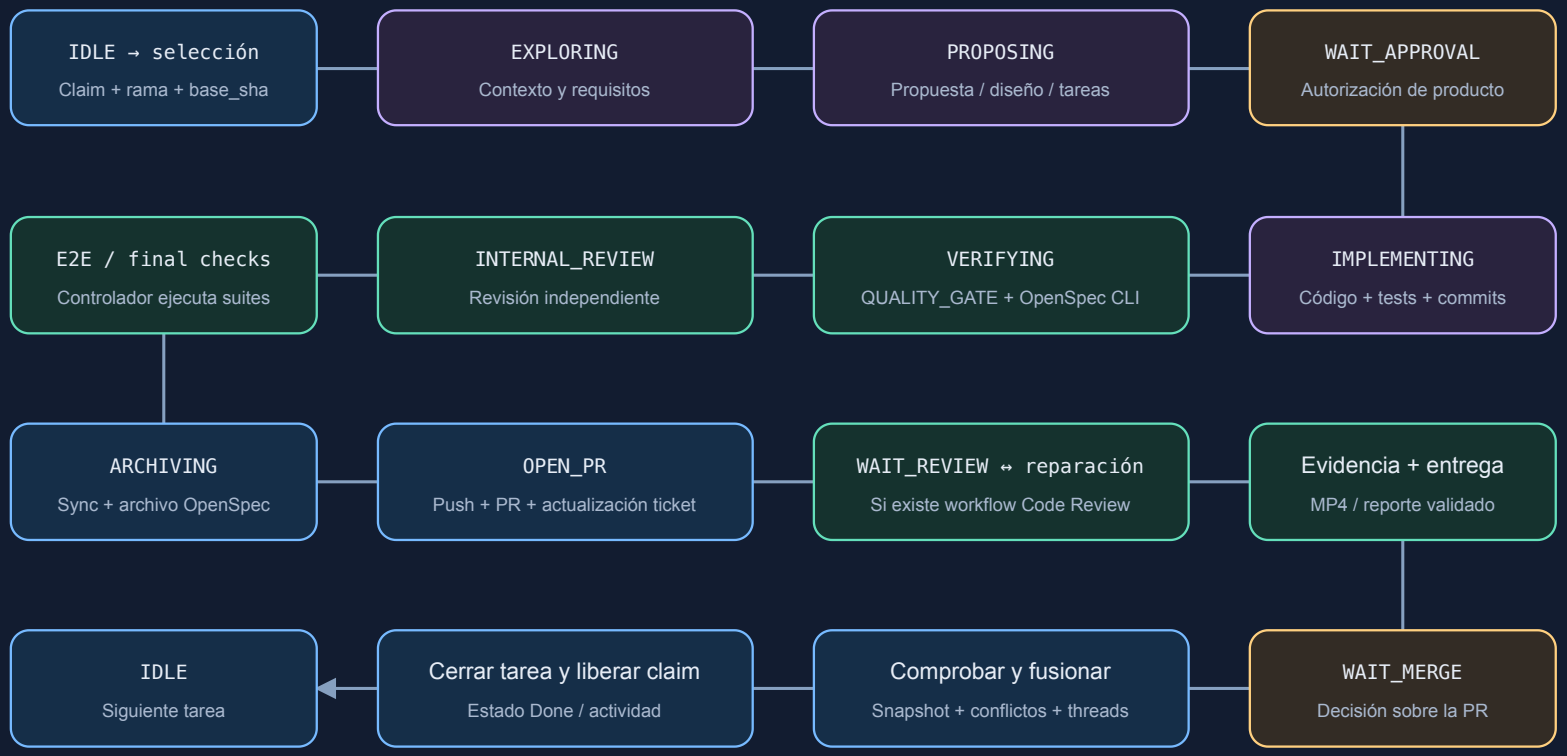
Verificación independiente

Codebot valida el cambio, consulta progreso CLI y contrasta checklist con QUALITY_GATE.

El controlador archiva, valida specs, hace push y abre la PR.

Ni OpenSpec ni Superpowers autorizan publicación por sí solos. El agente **devuelve control** al supervisor.

Una tarea avanza mediante puertas explícitas



RUTAS ALTERNAS: WAIT_REPLY / WAIT_STUCK · REPAIR_CHECKS · REPLANNING · RESOLVE_CONFLICTS · PUSHING

No son atajos de aprobación: conservan el trabajo y vuelven a las puertas que corresponden.

Preguntas, revisión, reparación y conflictos son rutas de retorno, no atajos de aprobación.

Las fronteras que no se deben confundir

Propuesta → código

Hace falta autorización de la versión del acuerdo. Una aclaración o un «continúa» ambiguo no bastan.

Código → PR

Checklist completo, calidad, revisión interna y controles del controlador.

PR → lista para revisar

Evidencia actual y entregable. Si hay harness, no se anuncia éxito con cero vídeos.

Decisión → merge

Estado, snapshot revisado, conflictos y threads; autorización humana inequívoca.

06 / ESTADOS DE LA FSM

24 estados registrados; retornos por contexto

18 fases

Trabajo, revisión, recuperación y publicación despachados por PHASES.

6 esperas

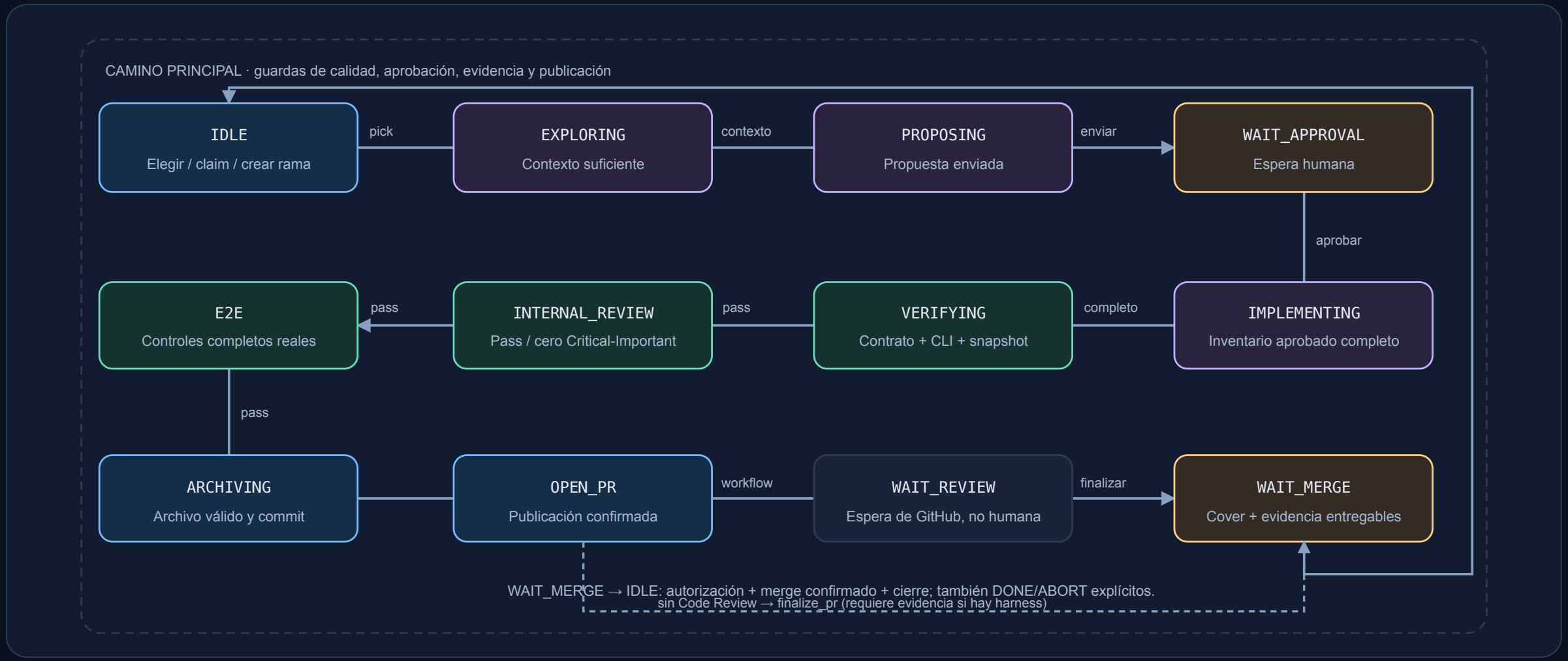
Aprobación, merge, respuesta, bloqueo, checkout limpio y review externa.

Contexto durable

return_state, stuck_return y push_context determinan la continuación.

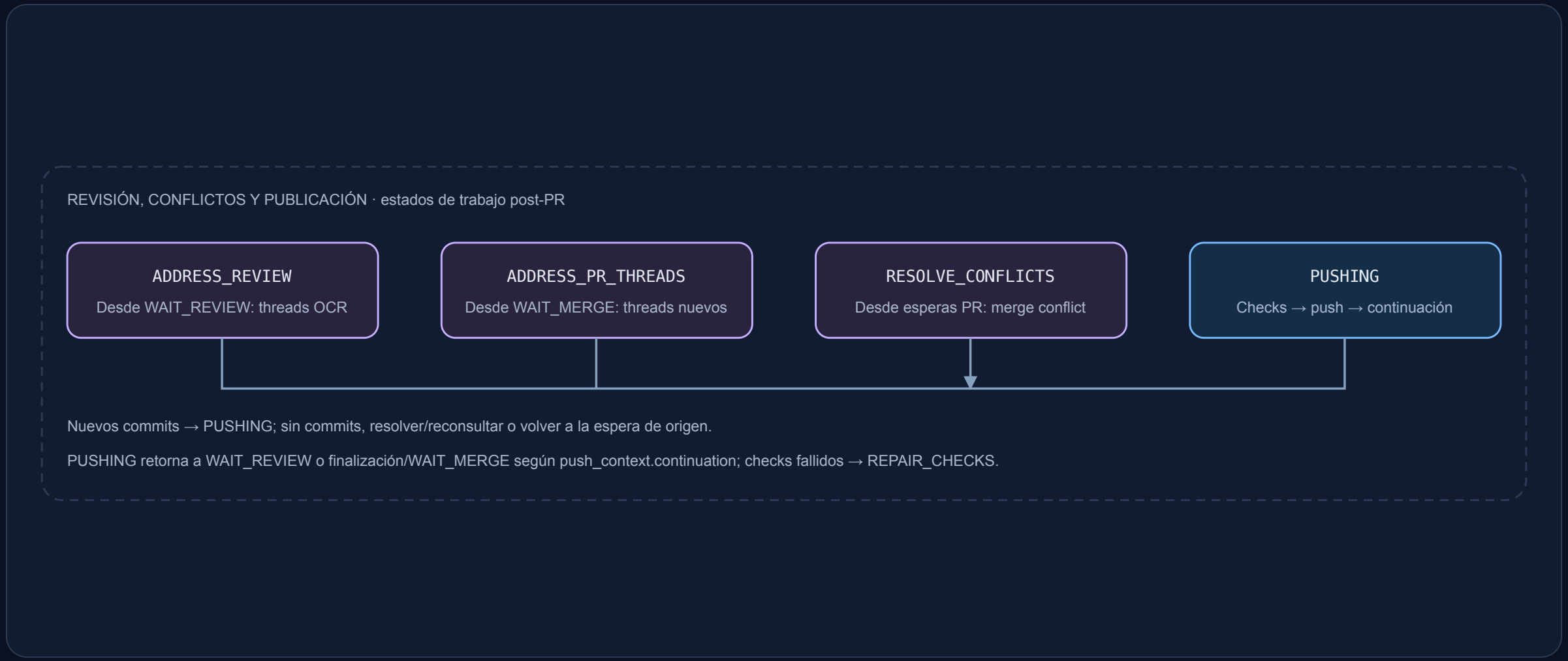
HOLD guarda una tarea aparcada; no es un estado del dispatch. FEEDBACK_QUESTION es una continuación, no una fase registrada.

FSM · Camino principal y guardas



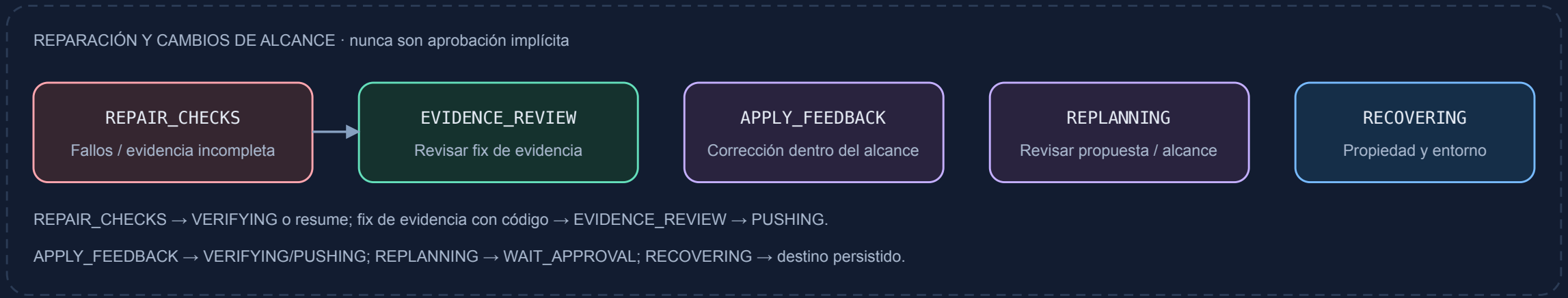
VERIFYING puede reutilizar una revisión vigente del mismo snapshot; ausencia de Code Review omite solo la espera externa.

FSM · Review, conflictos y publicación



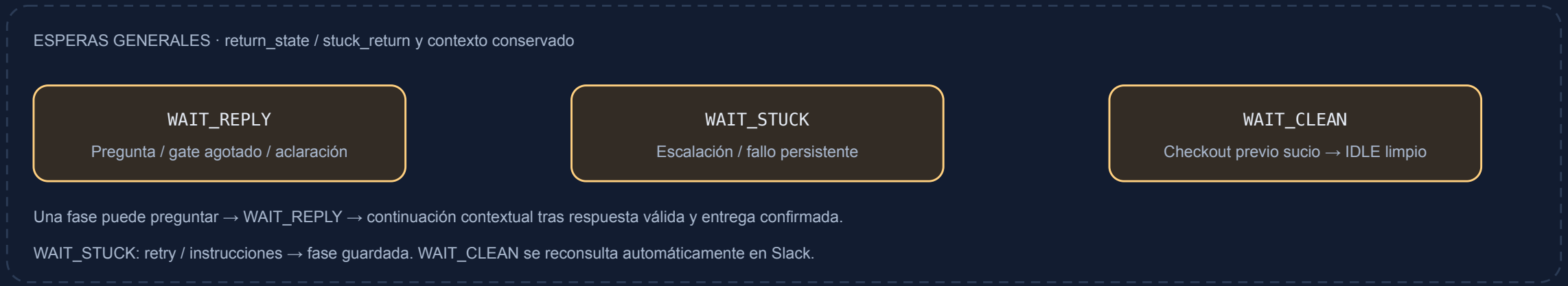
PUSHING conserva output, adjuntos y continuación; un push fallido no debe repetir el turno de reparación del agente.

FSM · Reparación y cambios de alcance



Corrección dentro del alcance ≠ replanning. La reparación de evidencia modificada exige revisión y controles antes de publicar.

FSM · Esperas y retorno seguro



Los límites agotados mantienen el trabajo bloqueado y presentan opciones concretas; no inventan un gate pass.

Estructura del sistema · Identidad y ejecución



La tarea agrupa intentos y resultados por task_id. El estado de ejecución vive en state.json; no todo objeto es una tabla.

Estructura del sistema · Resultados y entrega

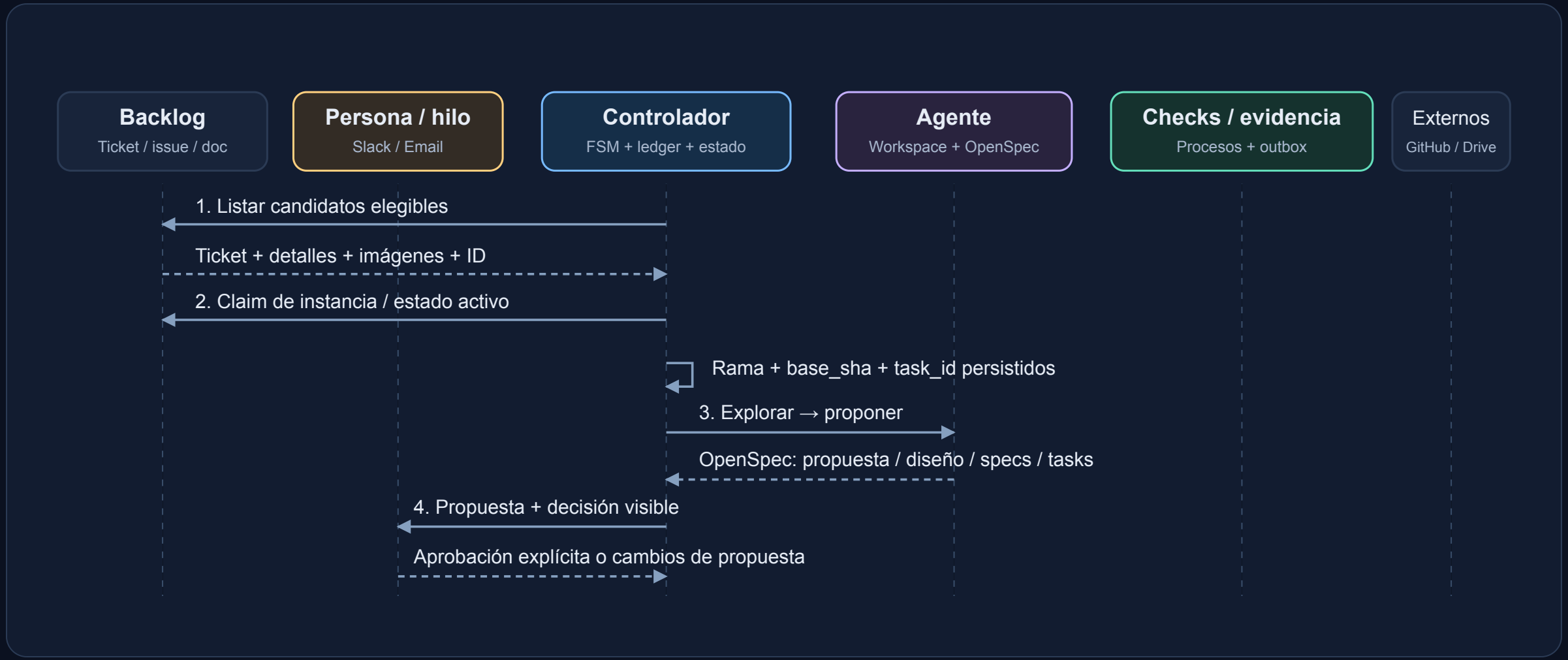


Relaciones conceptuales: una tarea agrupa múltiples intentos, checks, mensajes, artefactos y pasos de entrega por task_id.

Cada tabla del ledger comparte id, task_id, parent_id, identity, status, timestamps y data JSON; no es un esquema con FKs dibujadas aquí.

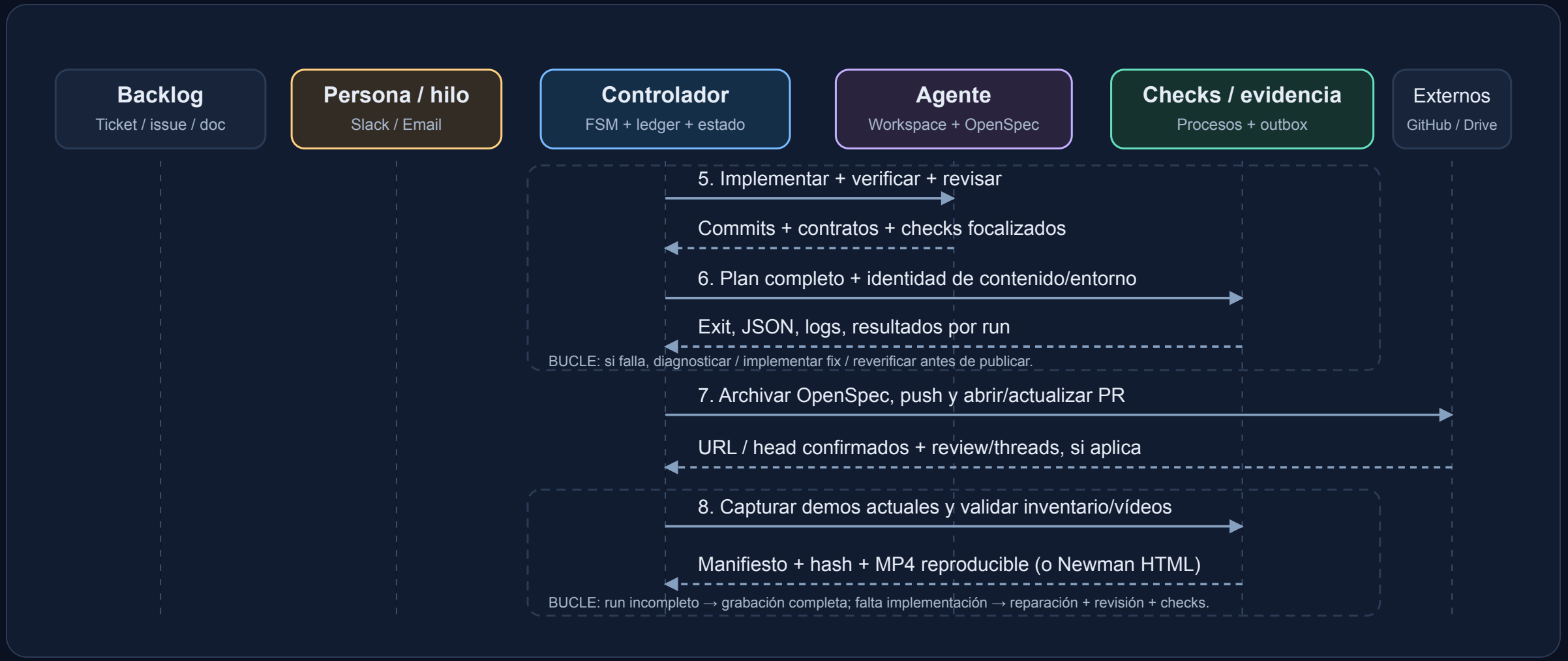
Relaciones conceptuales, no un esquema ER literal. Los registros SQLite comparten identidad, estado y data JSON.

Secuencia · Del ticket al acuerdo aprobado



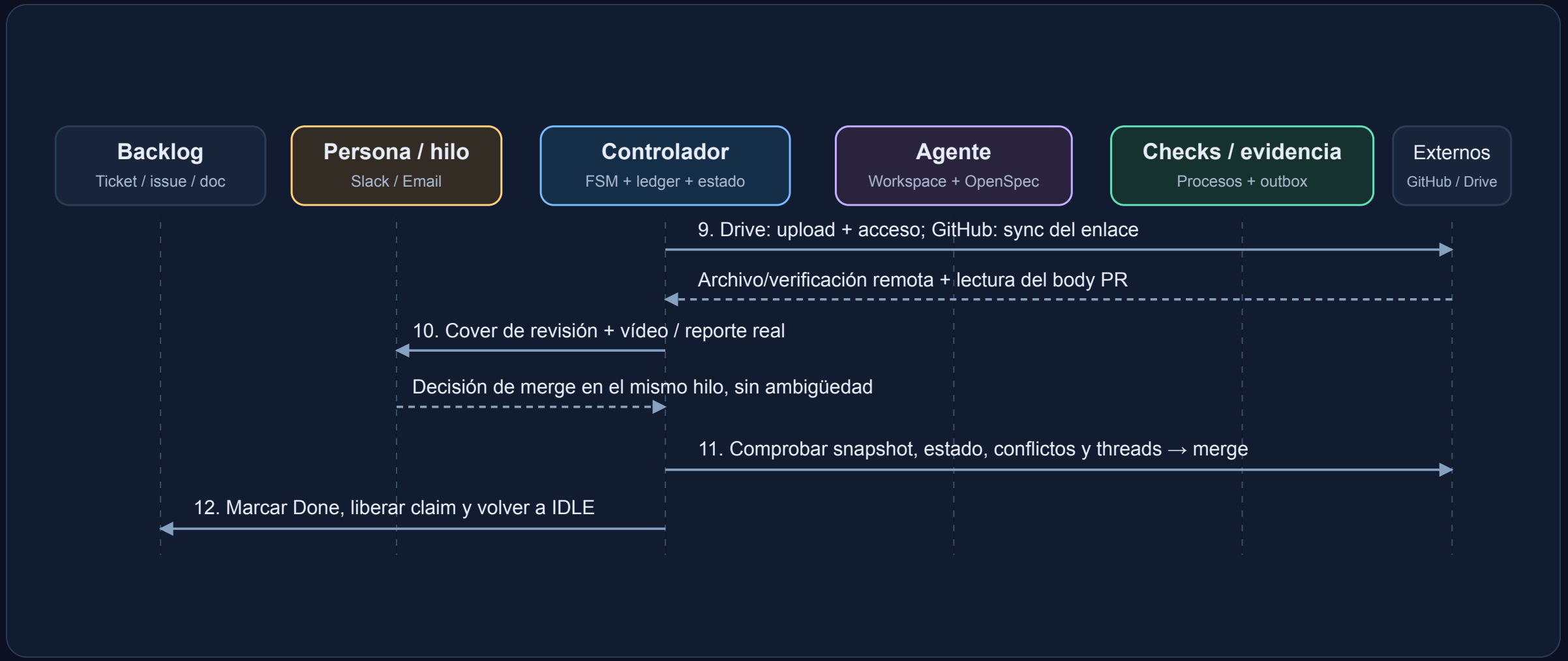
El tiempo avanza hacia abajo. Flechas punteadas: resultados y confirmaciones, no nuevas órdenes.

Secuencia · De la implementación a la evidencia



Los fallos agregan ciclos de diagnóstico, implementación y revalidación antes de publicar resultados.

Secuencia · Publicación, decisión y cierre



Flujo de datos y flujo de control son distintos: recibir un vídeo o pedir su publicación no autoriza merge.

Conversar no debe secuestrar el workflow



El worker lateral puede responder y registrar contexto; solo el controlador consume instrucciones que cambian la FSM.

Decisiones versionadas, respuestas completas

No inferir autorización

Se consideran texto íntegro, condiciones y negaciones.

Una selección antigua no autoriza una nueva pregunta. /btw siempre es lateral.

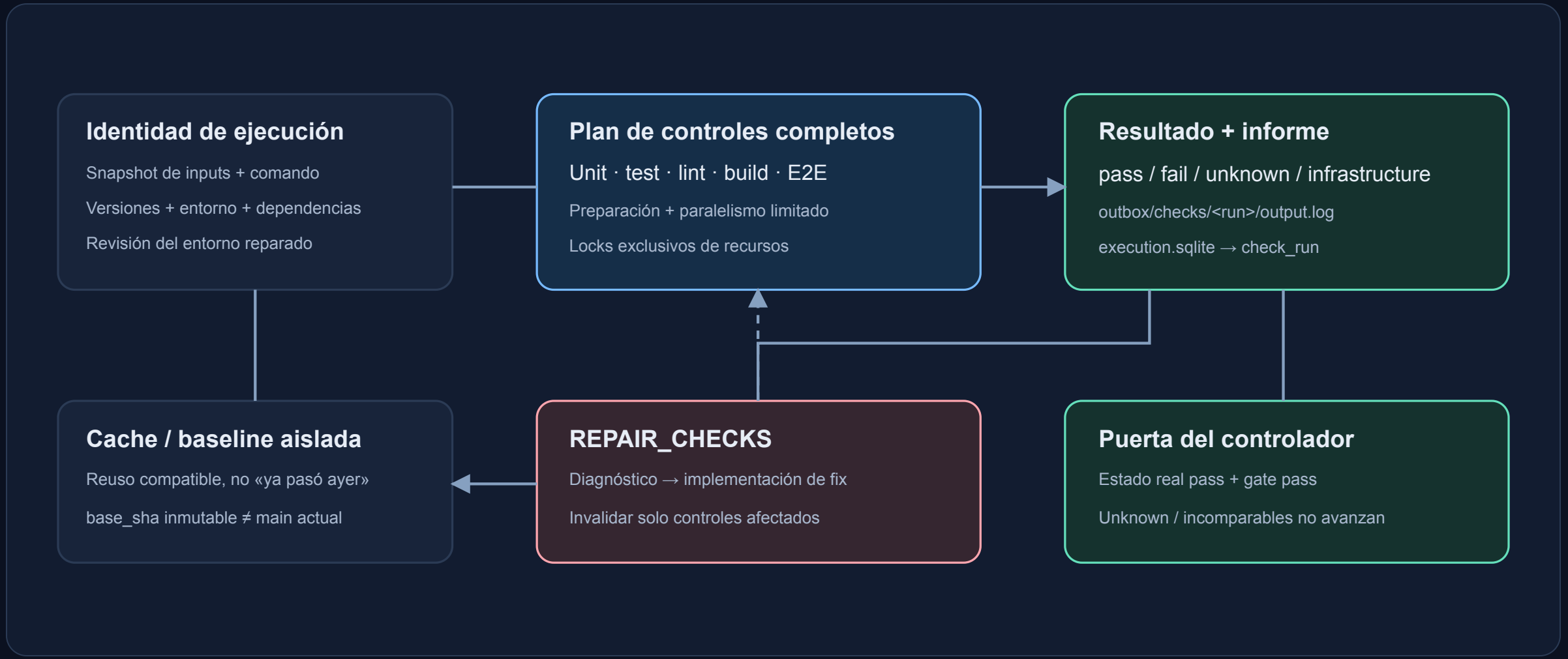
No perder respuestas

La respuesta y su continuación se guardan antes de entregar.

Si falla Slack/Email, reintentar transporte; no volver a generar el mismo resultado.

NEED_USER_INPUT pausa con opciones visibles y fase de retorno conservada. Solicitar evidencia no cambia una decisión de producto pendiente.

La prueba no es el resumen del agente



La política actual exige estado real pass y gate pass. Una baseline atribuye fallos; no convierte tests fallidos en éxito.

Checks reutilizables, pero nunca obsoletos

Identidad conservadora

Inputs de contenido + comando + herramientas + dependencias + entorno.

Reusar por «ya pasó ayer» no prueba la versión integrada.

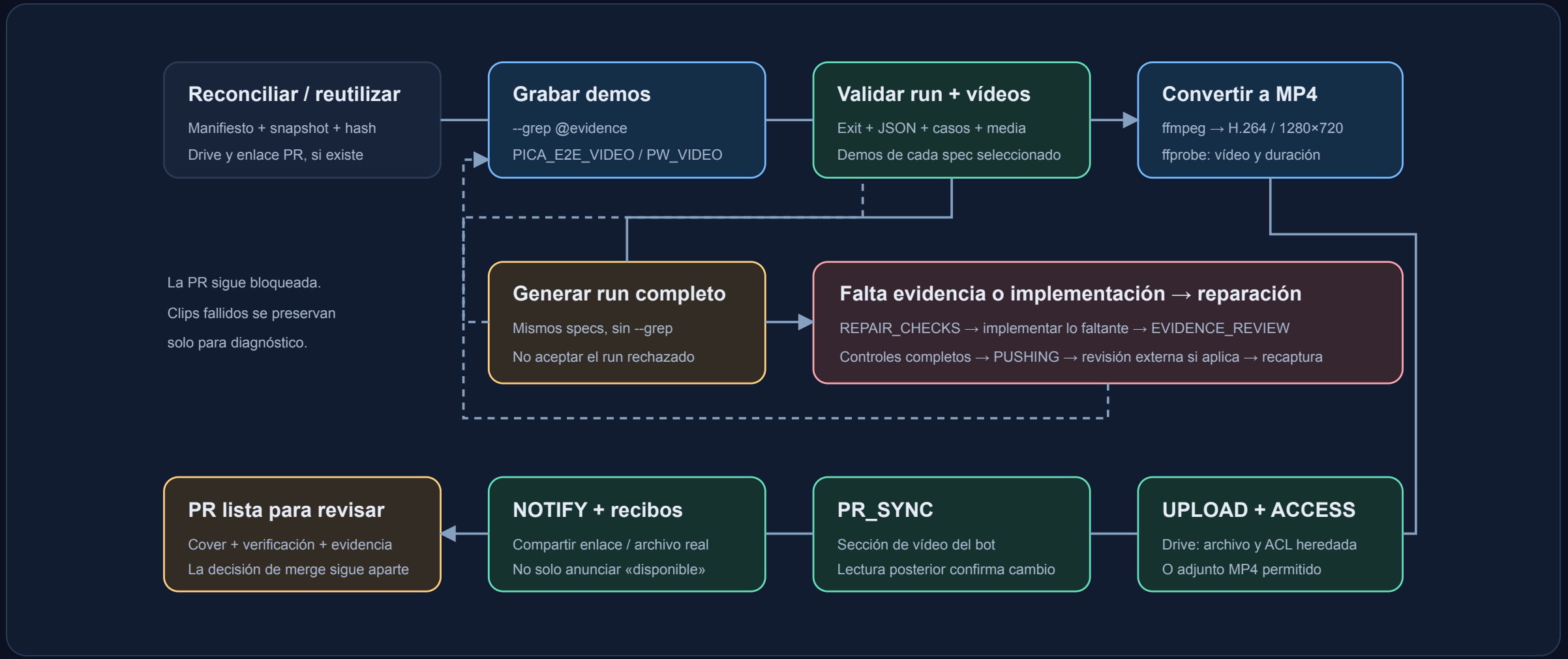
Reparación autónoma

Diagnosticar logs reales y corregir código, tests, intérprete, dependencias o servicios.

Invalidar checks afectados; reusar solo resultados vigentes.

Unknown, infraestructura y timeout **no pasan** por una afirmación del agente ni por «ignóralo».

Un WebM existente todavía no es evidencia entregada



Playwright produce vídeo UI; Newman puede producir un reporte API. Artefactos en outbox, no en Git del producto.

Si falta evidencia, producirla o implementar el fix

Run filtrado rechazado

Generar los specs completos con vídeo y validar el nuevo run.

No aceptar el exit fallido ni mezclar clips rechazados con evidencia aprobada.

Implementación incompleta

Crear demos, soporte de grabación o funcionalidad faltante dentro del alcance.

Revisión, controles y recaptura antes de anunciar la PR como lista.

No es una salida admisible: **«vídeos inválidos; entrego cero adjuntos»**.

Captura válida y publicación válida son puertas distintas

Procedencia + media

Run actual, JSON aprobado, inventario completo, vídeos únicos, hash y snapshot.

MP4 reproducible: duración y stream de vídeo comprobados con ffprobe.

Entrega confirmada

Drive verifica archivo y acceso heredado; no crea acceso público.

PR_SYNC conserva el body ajeno; lectura posterior confirma el enlace. Recibos confirman mensaje y adjuntos.

Si falla el upload o PR_SYNC, se reusa el MP4 válido; no se regraba por un fallo exclusivamente de transporte.

Estado de ejecución, ledger y Git cumplen funciones distintas



No resetear trabajo. No inventar éxito. No duplicar efectos externos inciertos.

Recuperar no significa empezar otra vez

Sesión o proceso interrumpidos

Reconstruir contexto desde Git, OpenSpec y checkpoints; conservar trabajo válido.

KICK cancela el turno, no cambia alcance ni descarta archivos.

Entrega parcial

Reconciliar recibos y reintentar solo lo no confirmado.

SQLite y APIs externas no forman una transacción: hay incertidumbre, no exactly-once universal.

Conservar **state.json, SQLite, Git, manifiestos y media**. No borrar historia para «forzar retry».

Un supervisor en Docker, stacks E2E en el host



CODEBOT_REPO_PATH conserva la misma ruta absoluta host/contenedor; el socket apunta al daemon del host.

Autonomía operativa no equivale a sandbox

Runtime confiable y privilegiado

Docker socket, Git, herramientas y proveedor tienen acceso configurado.

No ejecutar tareas no confiables suponiendo aislamiento OS fuerte.

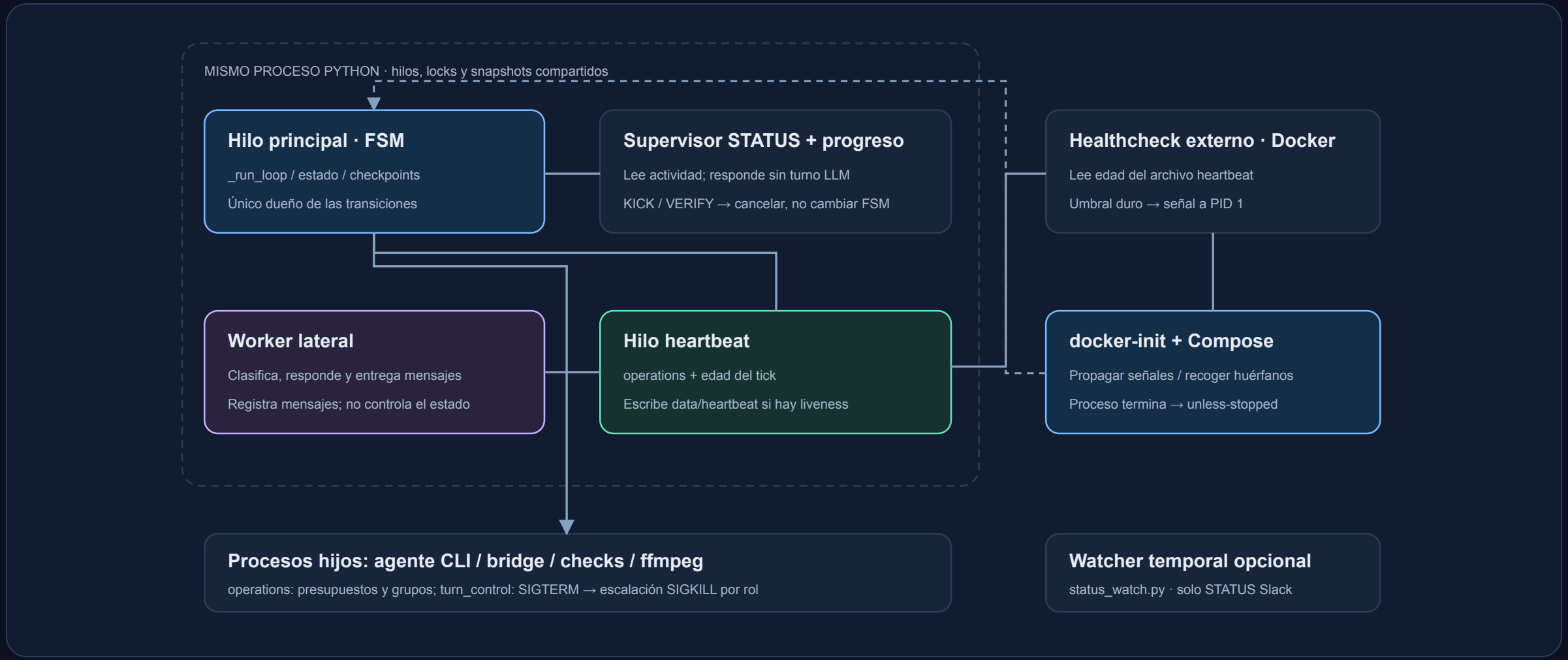
Presupuestos explícitos

Agente 1.800 s; E2E 900 s; tool local 45 s.

Subagente 600 s; proveedor 180 s; utilidad 30 s. Límites configurables y acotados por presupuesto padre.

Proteger .env, tokens, sesiones y datos. Reiniciar para aplicar configuración; cambios de toolchain pueden requerir rebuild.

Quién observa al proceso principal



Los hilos internos no son servicios independientes. La capa Docker puede detectar una señal obsoleta aunque Python deje de progresar.

Supervisión interna y externa tienen límites diferentes

Dentro de Python

STATUS y progreso: snapshots y comandos sin turno LLM; no mutan la FSM.

Heartbeat: renueva solo con liveness válida.
Worker lateral: conversación, no watchdog.

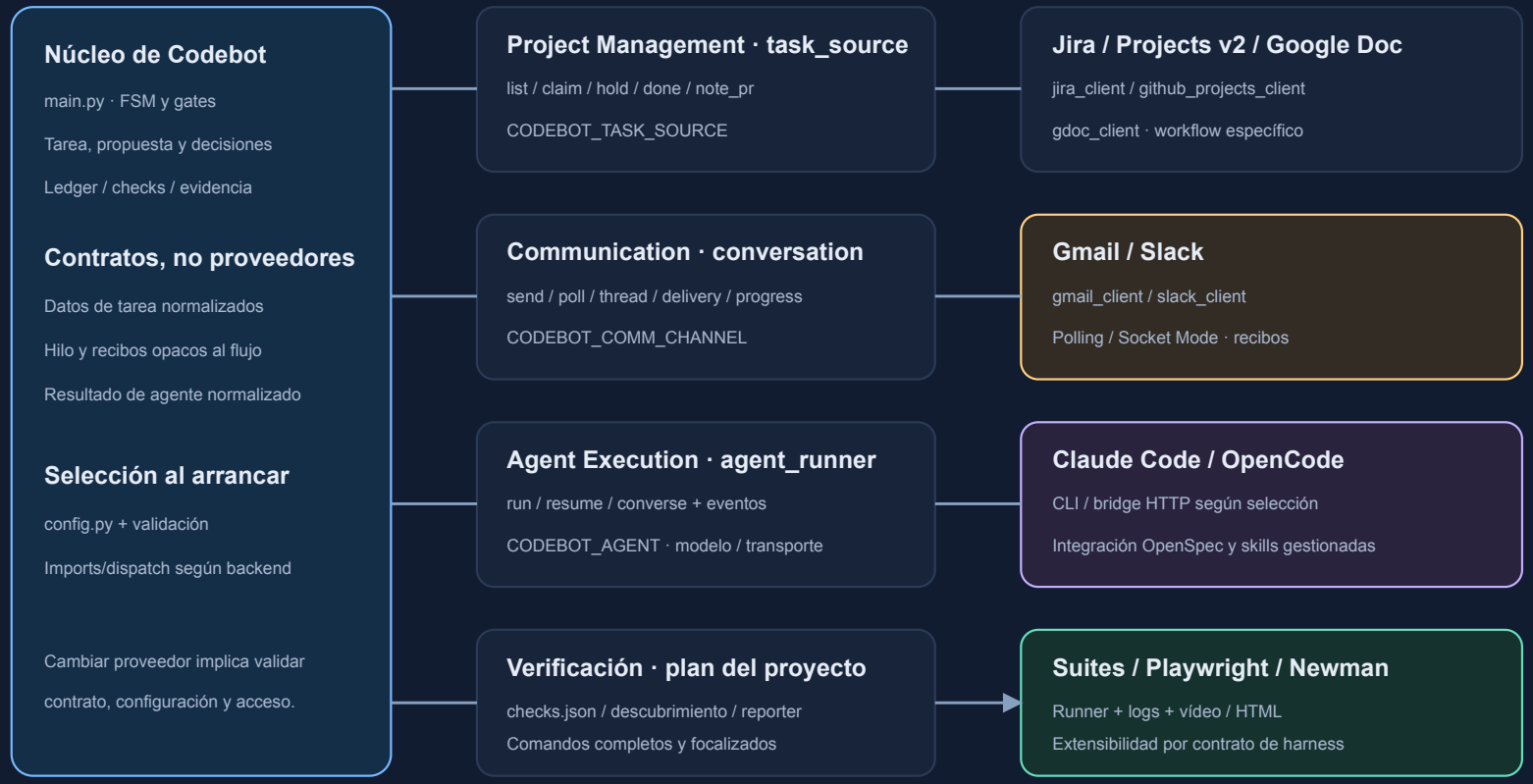
Fuera del hilo principal

Healthcheck Docker: edad de heartbeat; umbral duro fuerza salida y restart.

docker-init propaga señales y recoge huérfanos.
status_watch.py es fallback opcional de STATUS Slack.

Unhealthy no provoca restart por sí solo. operations y turn_control acotan procesos y cancelan grupos con SIGTERM/SIGKILL.

Project Management y Communication son independientes



Integraciones concretas actuales: Git/gh → GitHub PRs; drive_client → Google Drive. No hay adaptador GitLab/S3 implementado.

Puertos y adaptadores mediante funciones/dispatch Python; no un framework universal de plugins para todas las APIs.

Contratos comunes, capacidades diferentes

task_source.py

Jira / Projects v2 / GDoc.

Listar, claim, hold, done, activity y enlazar PR. Normaliza intención, no workflows idénticos.

conversation.py

Gmail / Slack.

Hilos, polling/eventos, envío, límites y recibos. Progreso editable es opcional.

agent_runner.py

Claude / OpenCode.

Run, resume, eventos, cancelación, contexto y resultado normalizado.

main.py importa conversation como gmail_client por compatibilidad histórica; ese alias no obliga a usar email.

Extender sin acoplar la FSM al proveedor

Implementar un adaptador

Contrato, selección, configuración y validación. Probar identidad, autorización, recuperación y efectos externos.

El backend traduce sus estados y capacidades; la FSM conserva su política.

Lo que no está implementado

GitHub/gh y Google Drive son integraciones concretas de código y publicación.

GitLab o S3 requieren diseñar un puerto nuevo; no aparecen por cambiar una variable.

Combinaciones existentes: jira + slack + opencode; github + email + claude; gdoc + slack + claude.

Comandos: preguntar, interrumpir y decidir no son lo mismo

Observar y controlar trabajo

`STATUS`: actividad y siguiente acción.

`KICK`: cancelar turno y retomar.

`HOLD / CONTINUE`: aparcar y reanudar.

Puertas explícitas

`VERIFY`: verificar tarea existente; no autoriza publicación.

`merge`: decisión de PR con gates.

`ABORT / DONE`: abandonar o cerrar; no simular merge.

Diagnosticar desde la evidencia, no desde el resumen

1. Identificar instancia, tarea, hilo, rama, PR y fase.
2. Leer transición y resultado del **controlador**.
3. Consultar logs y JSON del **mismo run**.
4. Comparar exit code, inventario y artefactos actuales.
5. Distinguir código, infraestructura, conversión, acceso y transporte.

«E2E pasó» y «la grabación posterior falló» pueden ser ciertos a la vez: **son ejecuciones y puertas diferentes**.

Dónde mantener el núcleo y sus integraciones

Orquestación y agentes

main.py · config.py · task_phases.py

agent_runner.py · claude_runner.py · prompts.py · agent-plugin/

Backlog y conversación

task_source.py · jira_client.py · github_projects_client.py · gdoc_client.py

conversation.py · gmail_client.py · slack_client.py · feedback.py · handoffs.py

Checks y evidencia

check_plan.py · checks.py · final_checks.py · check_baseline.py · check_repair.py

evidence.py · evidence_delivery.py · drive_client.py

Persistencia y runtime

execution_store.py · execution_identity.py · phase_checkpoint.py · delivery_checkpoint.py

operations.py · turn_control.py · scripts/ · Dockerfile · Compose

Contratos de agente; no órdenes arbitrarias

Preguntas y calidad

`NEED_USER_INPUT:` pregunta y espera.

`QUALITY_GATE:` calidad a validar.

`INTERNAL_REVIEW:` revisión limpia.

Evidencia y review

`E2E_SPEC:` specs de evidencia.

`ATTACH:` artefactos autorizados.

`RESOLVE:` threads tratados.

Los marcadores son interfaces verificadas. Su presencia no prueba éxito ni concede autoridad de publicación.

PRINCIPIO RECTOR

Conservar trabajo.
Verificar resultados.
Mantener la autoridad explícita.

Acuerdo

OpenSpec conserva lo aprobado.

Disciplina

Superpowers orienta al agente.

Control

Codebot supervisa; la persona decide.

El acuerdo guía el trabajo; la evidencia respalda los resultados; las decisiones humanas autorizan las acciones.